



ToF ReSTIR: Time-of-Flight Rendering with Spatio-temporal Reservoir Resampling

Juhyeon Kim, Wojciech Jarosz, Adithya Pediredla

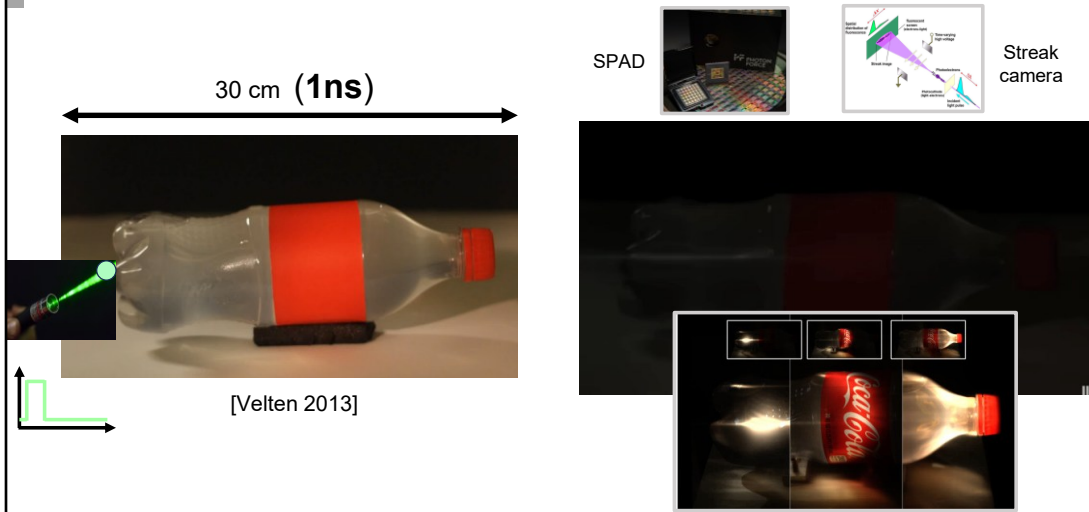
Dartmouth College



SIGGRAPH 2026
Los Angeles 19-23 JUL

Today, I will explain how ReSTIR techniques can benefit time-of-flight simulation, which is important in imaging community. Before talking about ReSTIR algorithm, I will first explain what Time-of-flight imaging is.

Light Travels Fast, but *not* Infinitely Fast



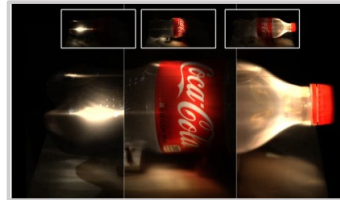
2

Suppose there a Coke bottle filled with water and we shine a pulse of light at the left side. As the light passes through the bottle, it takes a small but finite amount of time to travel. It will be one nanosecond for this coke bottle scale, which was long thought to be impossible to measure. However, recent advances in hardware now allow us to directly observe this time-resolved light transport slice-by-slice.

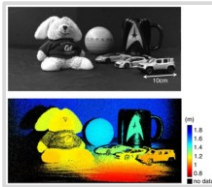
Time-of-Flight Imaging

Time-resolved measurements
(time-of-flight imaging)

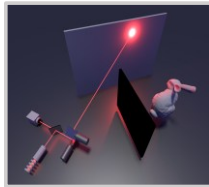
[Velten 2013]



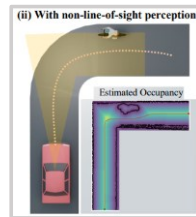
Applications



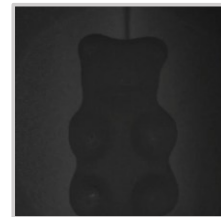
Depth Estimation
[Morimoto 2020]



NLOS Reconstruction
[O' Toole 2018]



NLOS Navigation
[Young 2025]



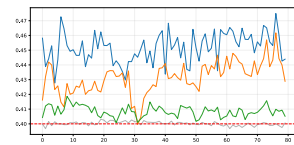
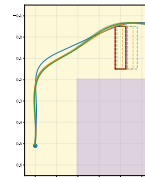
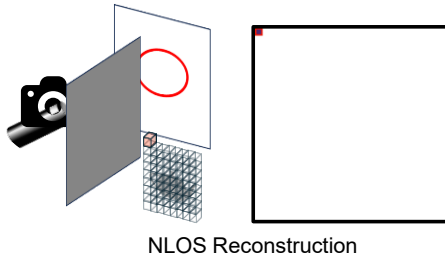
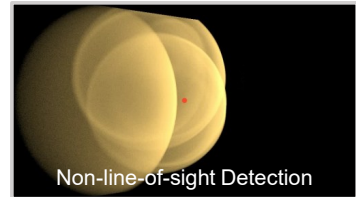
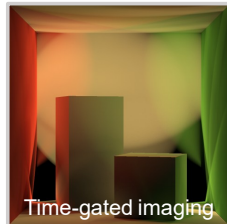
Light Transport Decomp.
[Gkioulekas 2015]

3

This **time-resolved light transport measurement**, or **time-of-flight (ToF) imaging**, enables a wide range of applications, including depth estimation hidden object reconstruction, navigation, and light transport decomposition.

Time-of-Flight Rendering

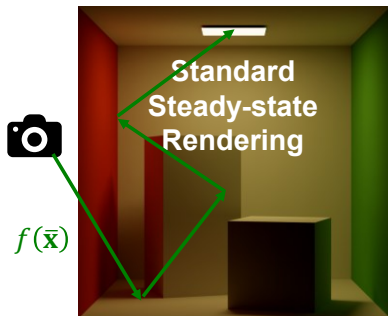
ToF
Rendering &
Simulation



NLOS Navigation

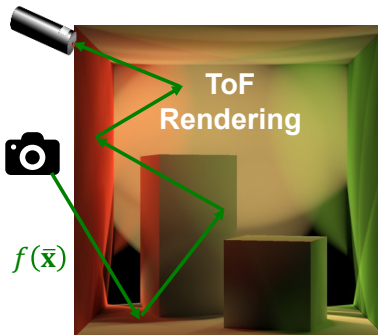
Time-of-Flight rendering aims to accurately simulate these measurements, providing a virtual testbed for developing or validating various ToF algorithms.

Standard Rendering vs ToF Rendering



$$I = \int_{\mathcal{P}} f(\bar{\mathbf{x}}) d\mu(\bar{\mathbf{x}})$$

Path
Throughput



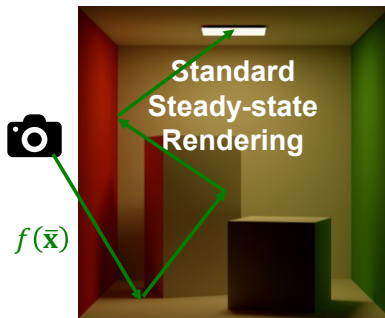
$$I(\tau) = \int_{\mathcal{P}} f(\bar{\mathbf{x}}) d\mu(\bar{\mathbf{x}})$$

Path
Throughput

5

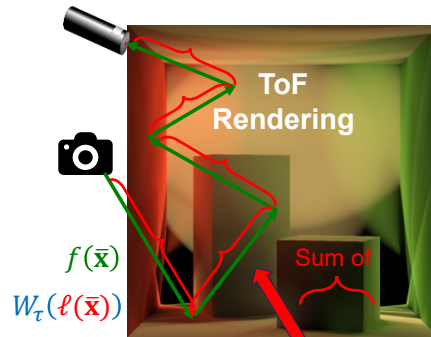
More formally, let's look at the path integral formulation. In standard rendering, we integrate the path throughput over all light paths, whereas in time-of-flight rendering,

Standard Rendering vs ToF Rendering



$$I = \int_{\mathcal{P}} f(\bar{\mathbf{x}}) d\mu(\bar{\mathbf{x}})$$

Path
Throughput



$$I(\tau) = \int_{\mathcal{P}} f(\bar{\mathbf{x}}) W_{\tau}(\ell(\bar{\mathbf{x}})) d\mu(\bar{\mathbf{x}})$$

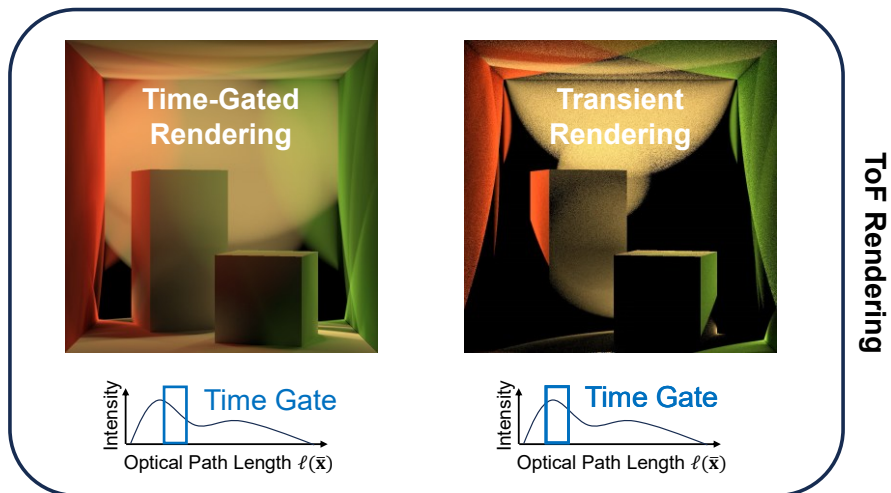
Path Throughput Path length Importance



6

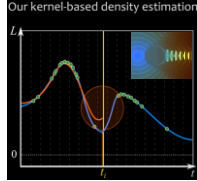
we additionally weight each path by a path-length importance function (click), which depends on optical path length $\ell(\mathbf{x})$ a sum of all path segment length. Path-length importance function often implemented as a box or time-gating function.

Time gated and Transient Rendering

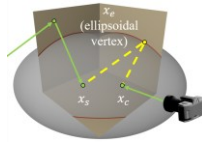


A fixed gate corresponds to time-gated rendering, while a sliding gate corresponds to transient rendering. Together, these are referred to as time-of-flight (ToF) rendering, which we aim to accelerate.

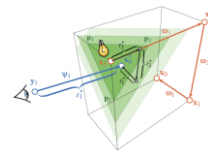
Previous Works on Time-of-Flight Rendering



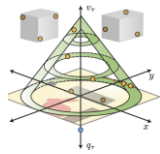
[Jarabo 2014]



[Pediredla 2019]



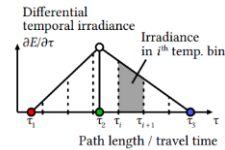
[Liu 2022]



[Cui 2026]



[Royo 2025]

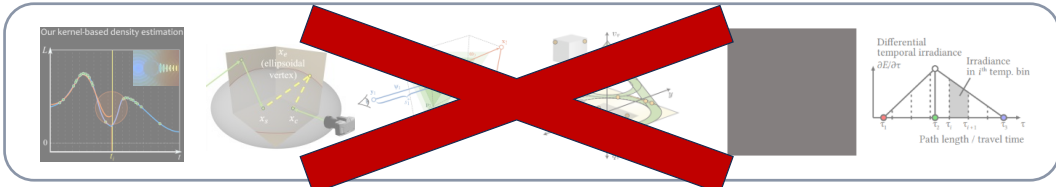


[Iseringhausen 2020]

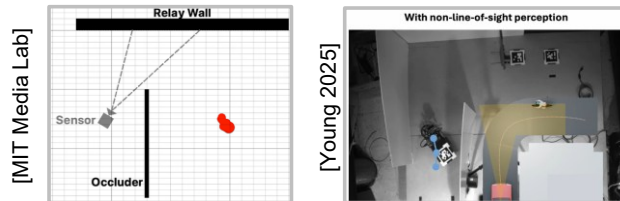
Offline rendering Restricted Config + Mesh Processing

Several time-of-flight rendering methods have been proposed, including kernel density estimation, ellipsoidal path connection, temporally sliced photon primitives, light-cone shells, and GPU-based path tracing or radiosity approximations. However, these methods either target offline rendering or rely on restricted scene configurations.

Previous Works on Time-of-Flight Rendering



Often takes **few minutes** to render a single image

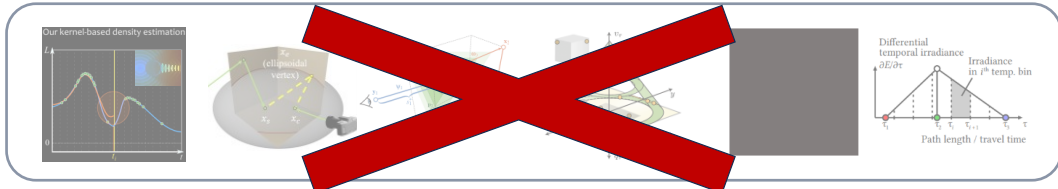


→ cannot be used for **interactive-rate** simulations (ms-scale)

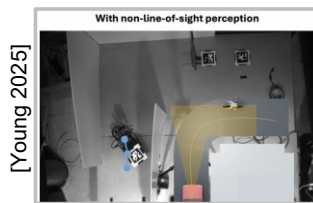
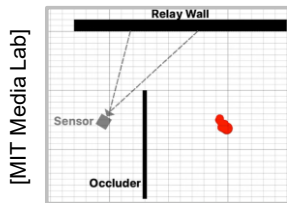
9

making them often unsuitable for several interactive downstream applications which requires strict latency constraint.

Previous Works on Time-of-Flight Rendering



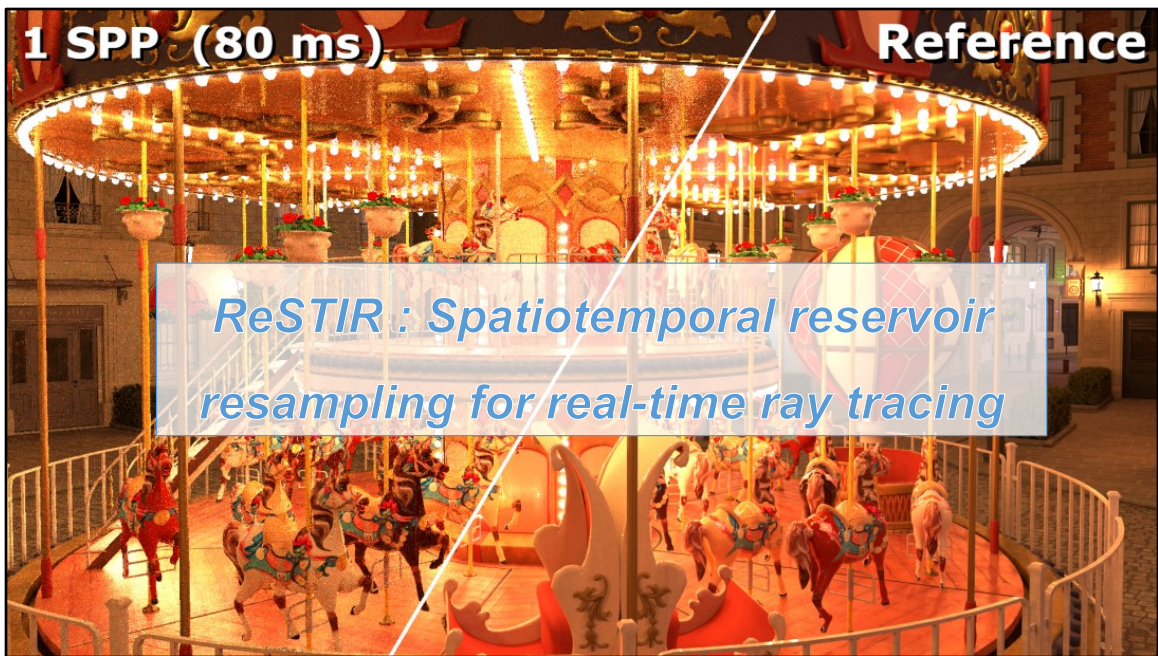
Often takes **few minutes** to render a single image



→ cannot be used for **interactive-rate** simulations (ms-scale)

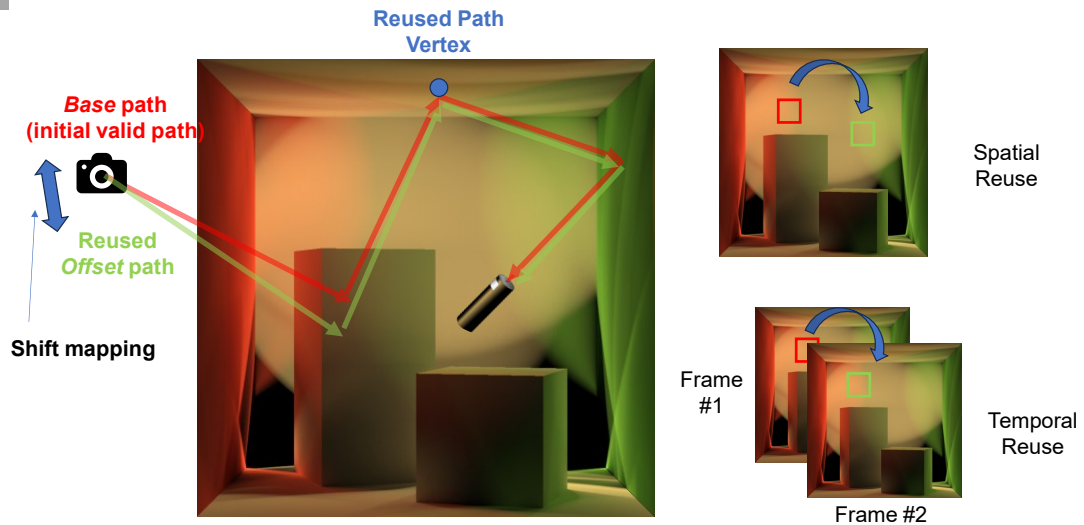
10

Instead, we draw inspiration from real-time rendering, which can make ToF simulation even run at interactive rate, which was not possible before.



And ReSTIR is a representative example that enables real-time path tracing.

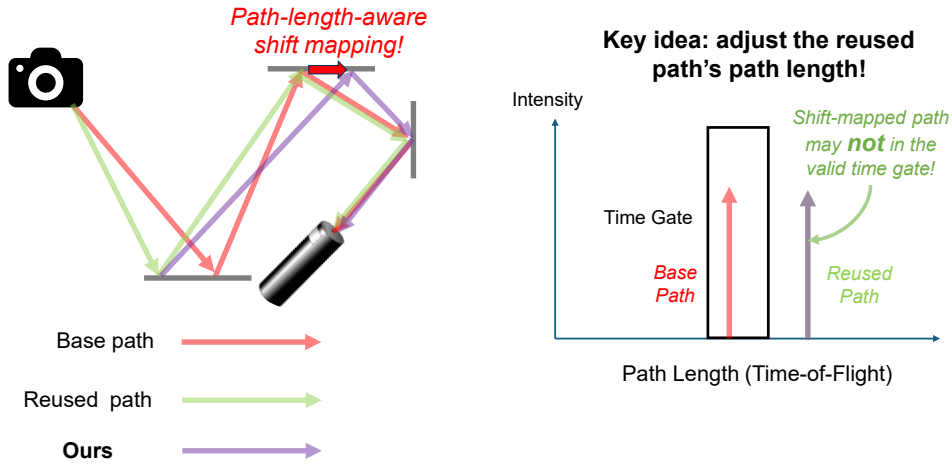
Path Reuse in ReSTIR



12

I will not go into the details of ReSTIR, but the key idea is simple: it reuses a base path that have been proven useful in one pixel, into other pixels as so-called offset paths. This reuse can happen either spatially or temporally across frames. The mapping between a two paths is referred to as *shift mapping*.

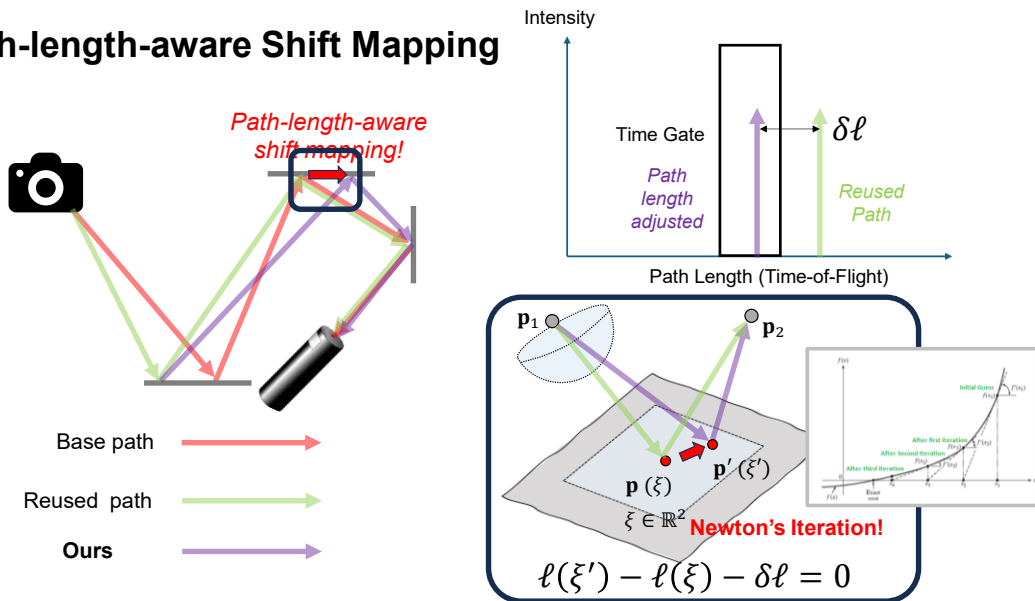
Problem of Steady-state Shift Mapping



13

However, when we directly apply existing shift-mapping strategies designed for steady-state rendering, the reused path may fall outside the valid time gate, making path reuse ineffective. To address this, we propose a specialized shift-mapping strategy that keeps the reused path temporally valid.

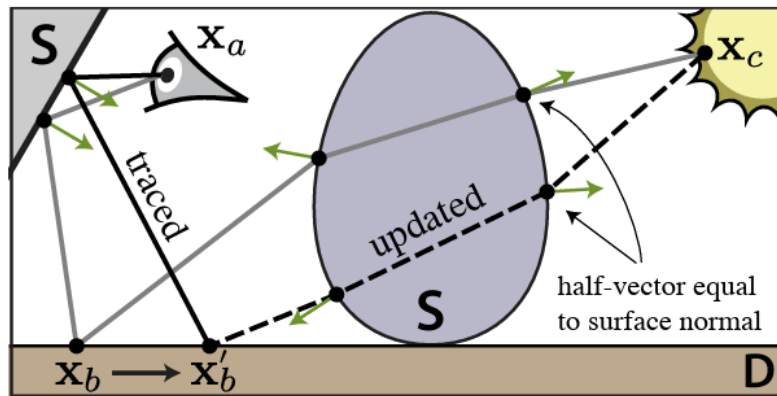
Path-length-aware Shift Mapping



14

So how did we achieve this? We first parameterize vertex point using a 2D coordinate ξ , which could be both local coordinate or polar coordinate on hemisphere. This reformulates our goal as a 2D **root-finding problem of ξ'** given amount of deviation $\delta \ell$, which we solve using **Newton's method**.

Specular vs Path-length Constraint

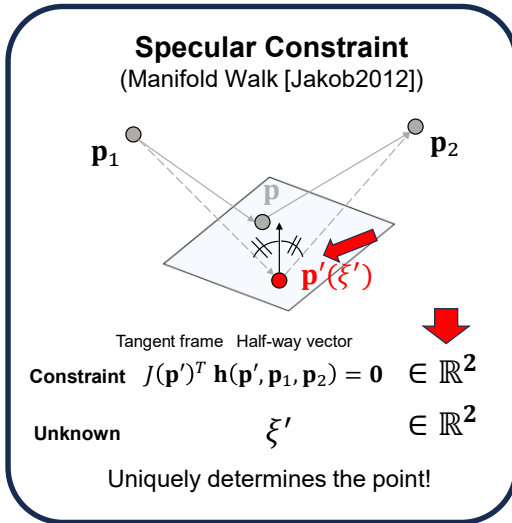


Manifold Exploration [Jakob2012]

15

In fact, this is closely related to handling **specular constraints** in caustics rendering.

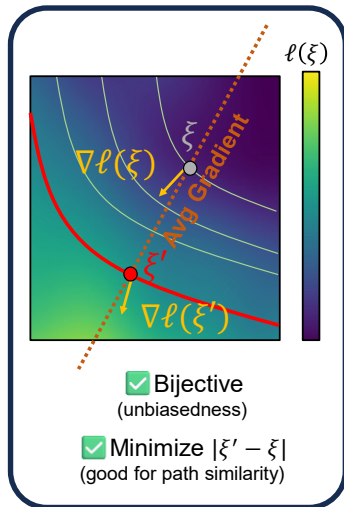
Specular vs Path-length Constraint



16

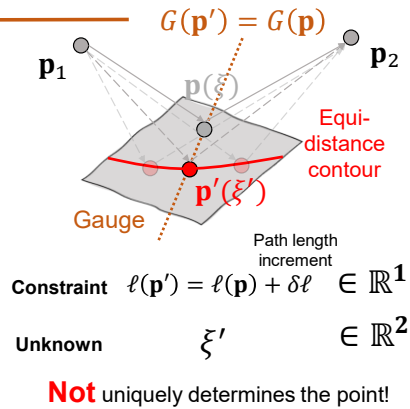
However, there is a crucial difference. For specular constraints, the condition is defined by specular reflection in the local tangent frame, resulting in a **2D constraint** that uniquely determines the new surface point.

Gauge Condition



Which gauge condition?

Path Length Constraint

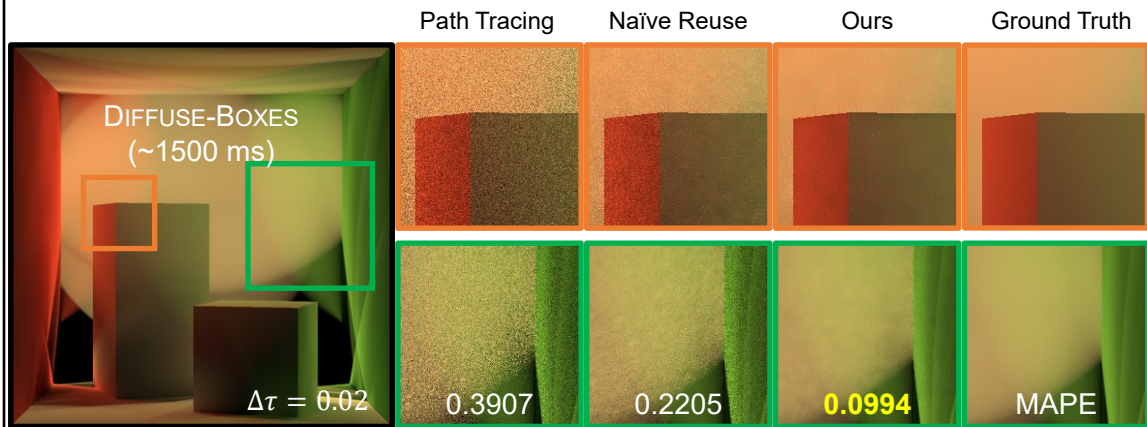


18

Several implementation could be possible, but we propose using the **average gradient direction**, which is bijective so guaranteeing unbiasedness and also minimizes displacement, which is desirable for path similarity.

Offline Rendering Demonstration (Spatial Reuse Only)

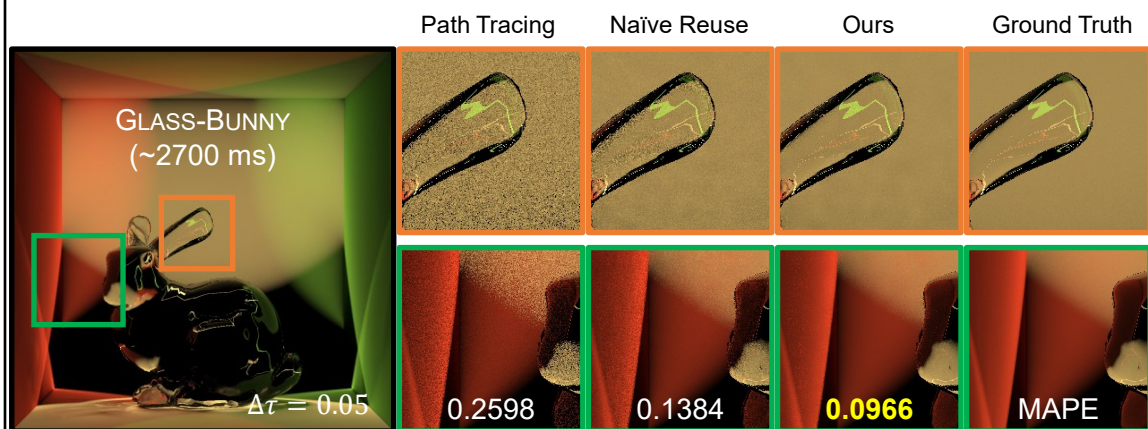
Offline Rendering Results



20

We first present offline rendering results on the Cornell Box scene with different materials. Compared to path tracing or naïve reuse, our method produces better results both qualitatively and quantitatively under an equal time budget.

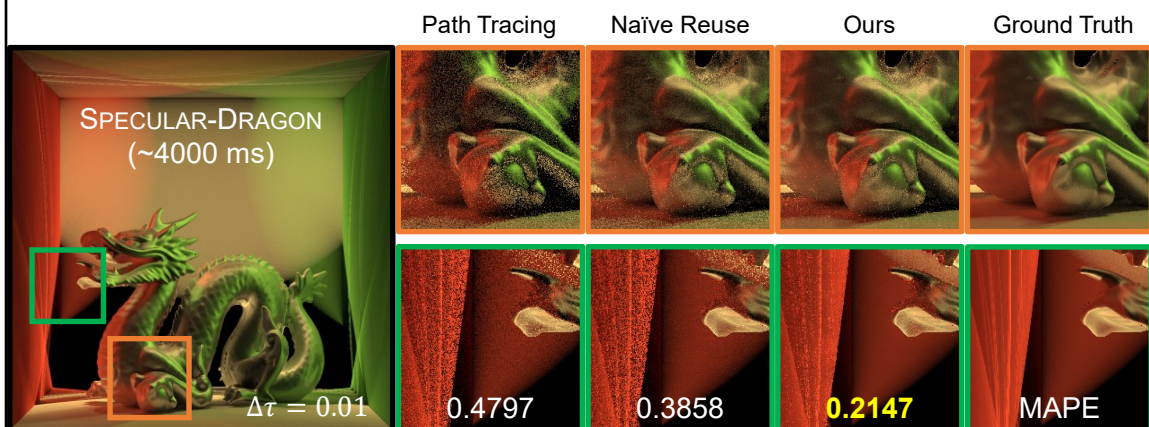
Offline Rendering Results



21

Here is another scene with glass bunny

Offline Rendering Results



22

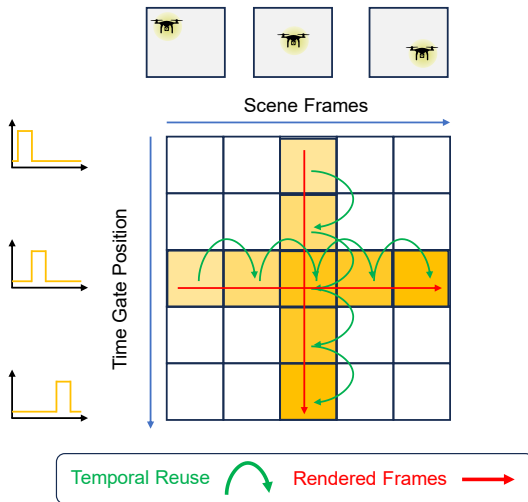
And specular dragon.

Real-time Rendering Demonstration (Spatial Reuse + Temporal Reuse)

23

Now, I also show real-time results with both spatial and temporal reuse.

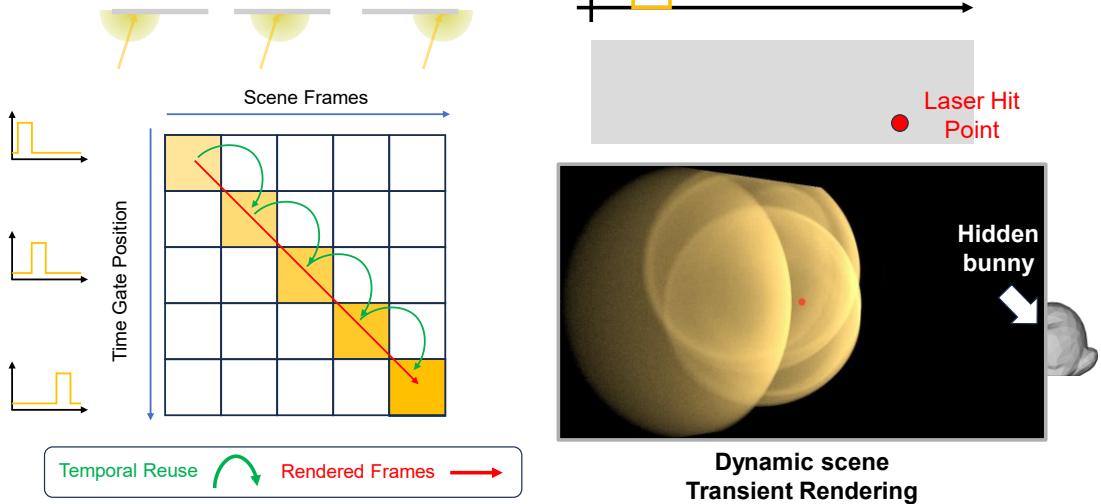
Real-time ToF Rendering



24

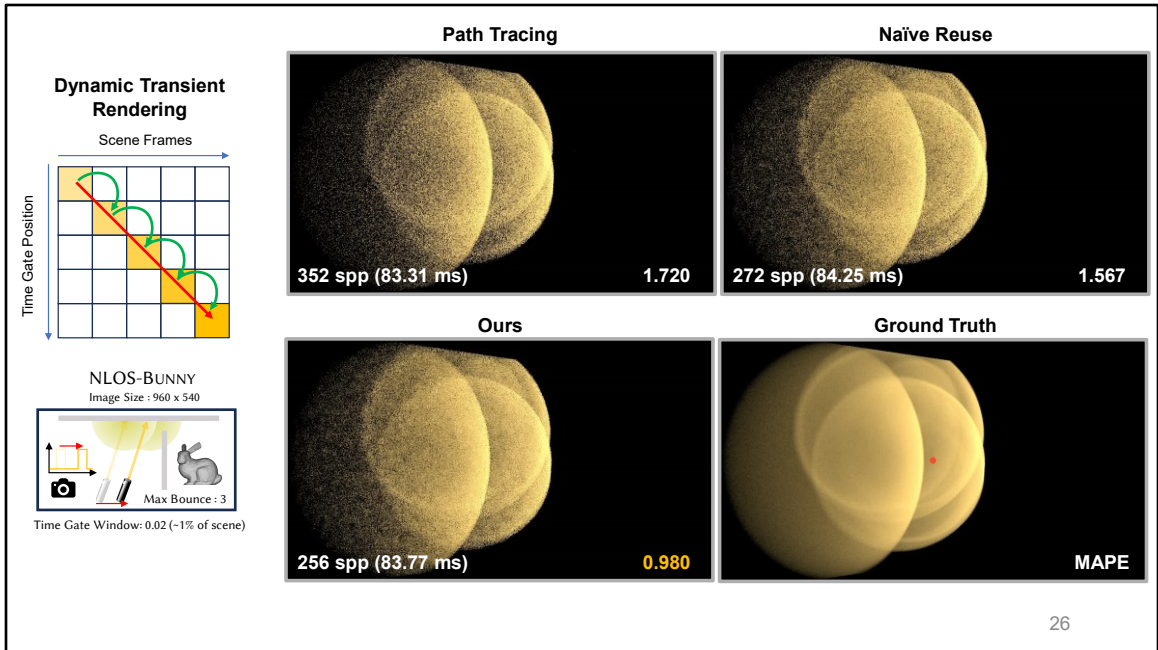
For time-of-flight rendering, frames can progress along two axes: scene evolution and time-gate shifting. Consequently, temporal reuse can also occur along these two axes. Vertical case corresponds to conventional transient imaging in static scenes, while horizontal case corresponds to time-gated imaging in dynamic scenes, such as proximity cameras.

Real-time ToF Rendering

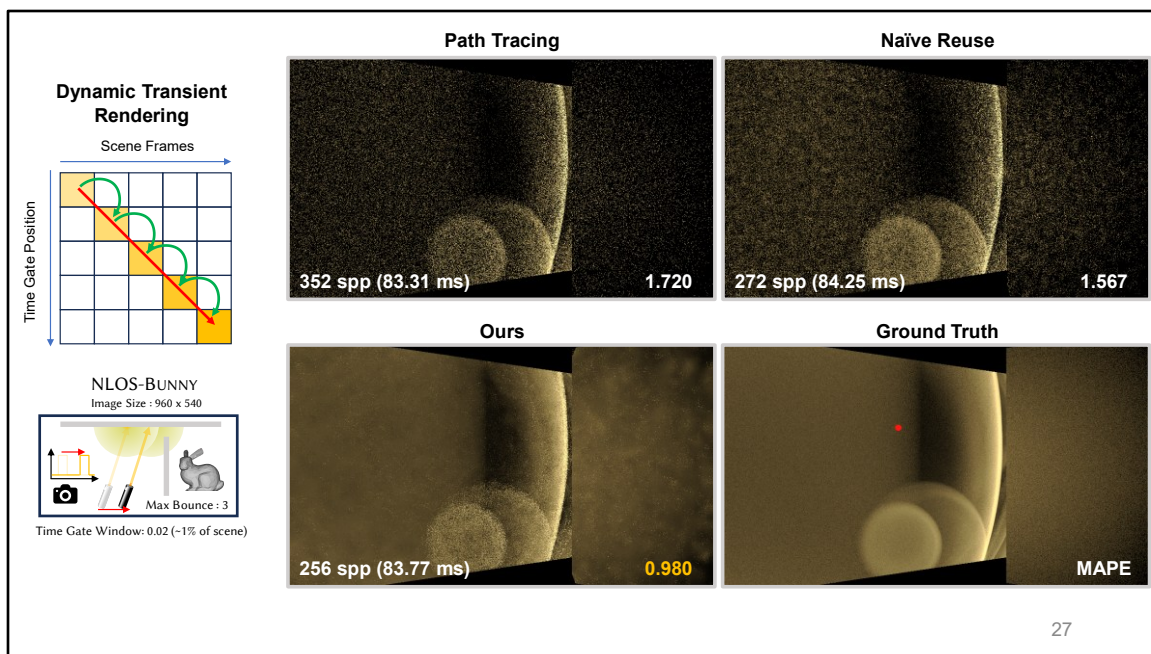


25

A more interesting case is a diagonal case, which happens constructing transient histograms with time-gated cameras under the dynamic scenes. (click) Here, we show an example of a hidden bunny scene while both the laser hit point and the time gate move simultaneously.



And as you can see, proposed method gives better result compared to baseline methods simulating this scenario.

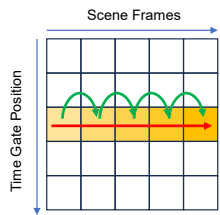


Here is an image version which more clearly shows the difference



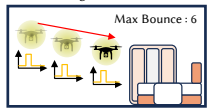
We could also see similar advantage in other reuse cases.

Dynamic Time-Gated Rendering

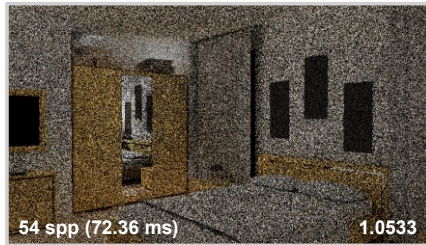


BEDROOM-HOVER

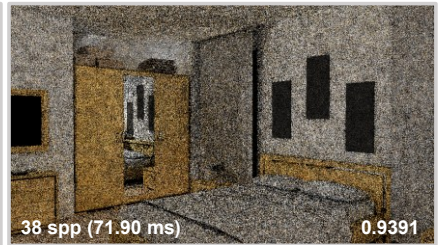
Image Size : 960 x 540



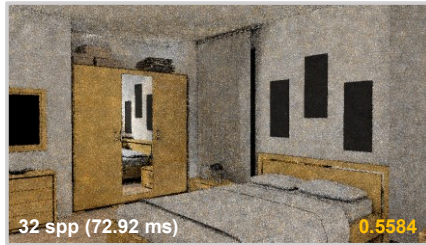
Path Tracing



Naïve Reuse



Ours

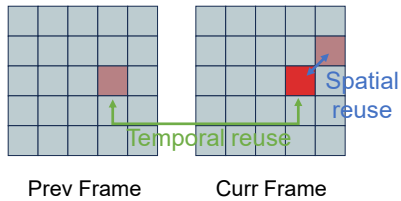


Ground Truth



Reuse in Histogram

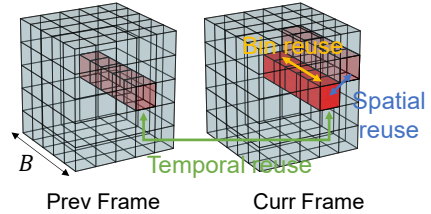
Reuse in Image (2D)



Per frame
output :

$$H \times W$$

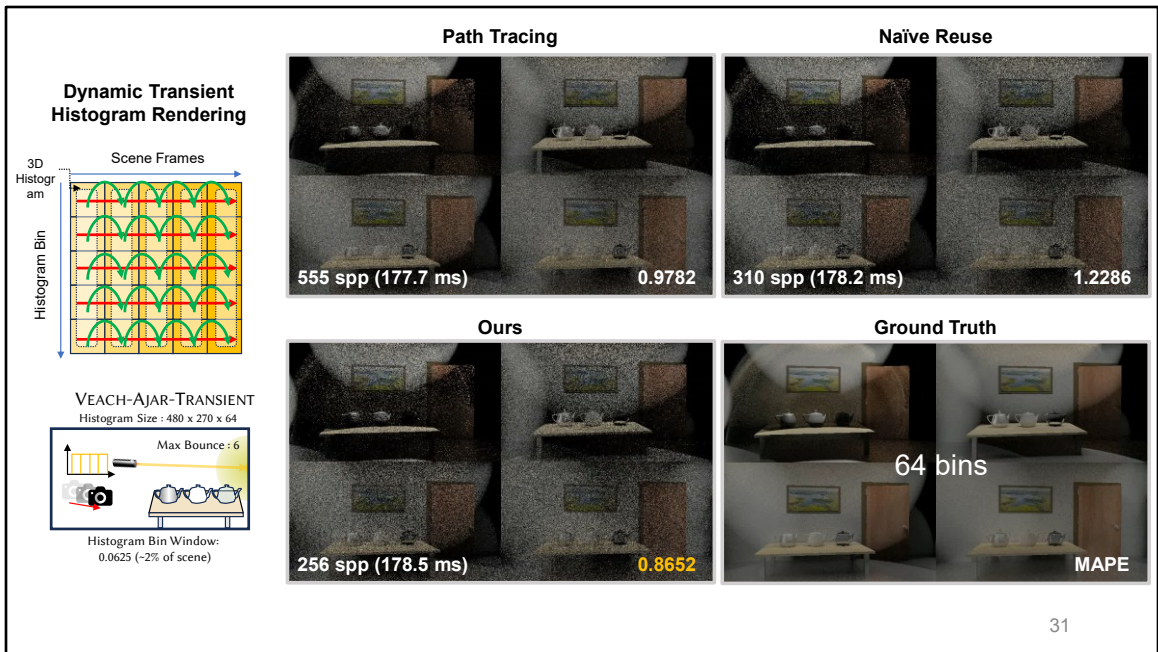
Reuse in Histogram (3D)



$$H \times W \times B$$

30

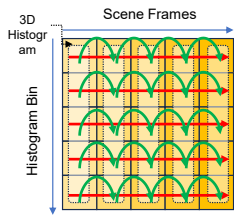
So far, we only considered reusing within image, but we can extend this to **per-frame transient histogram rendering, which now outputs 3D histogram per frame**



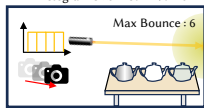
31

Since we need to apply reuse at each bin, reuse cost is higher, but we found proposed method gives better result with moderate number of bins.

Dynamic Transient Histogram Rendering



VEACH-AJAR-TRANSIENT
Histogram Size : 480 x 270 x 64



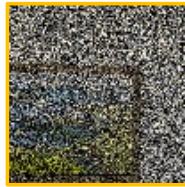
Histogram Bin Window:
0.0625 (~2% of scene)

Path Tracing



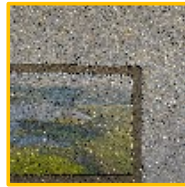
0.9782

Naïve Reuse



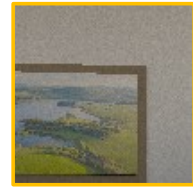
1.2286

Ours



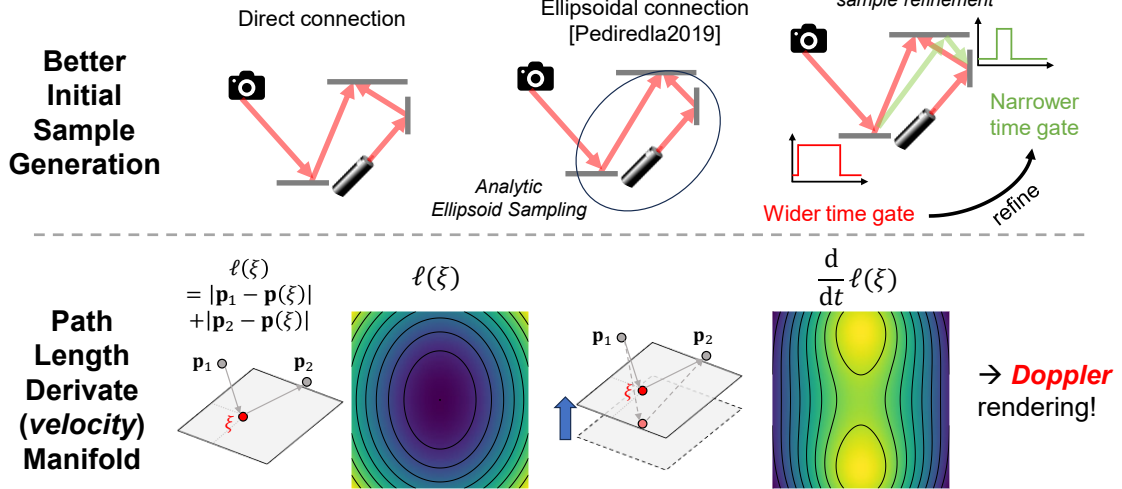
0.8652

Ground Truth



MAPE

Several Extensions (see papers for detail!)

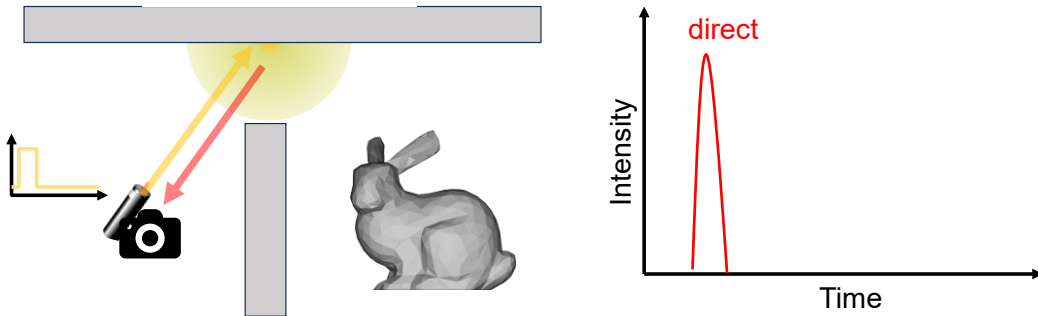


33

I will also briefly show two interesting extensions, where details are in the paper. First, our method can be combined with improved initial sample generation techniques, such as ellipsoidal path connection. We can also adapt our path-length-aware shift mapping into a shrink mapping, which refines samples from a wider, and therefore easier-to-sample, time gate. Second, while we focused on the path-length manifold, the same idea can be extended to the path-length derivative, or velocity manifold, enabling efficient simulation of the Doppler effect.

Downstream Applications

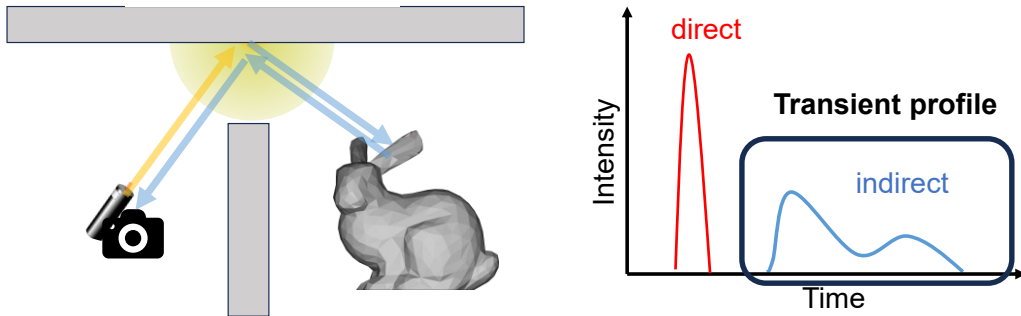
Non-line-of-Sight (NLOS) : *See around the Corner*



35

We demonstrate downstream applications of non-line-of-sight imaging, which aims to see around corners. Let me briefly explain how it works. We first shoot pulsed laser onto the back wall. Part of the light reflects directly from the wall, creating this peak in the transient profile.

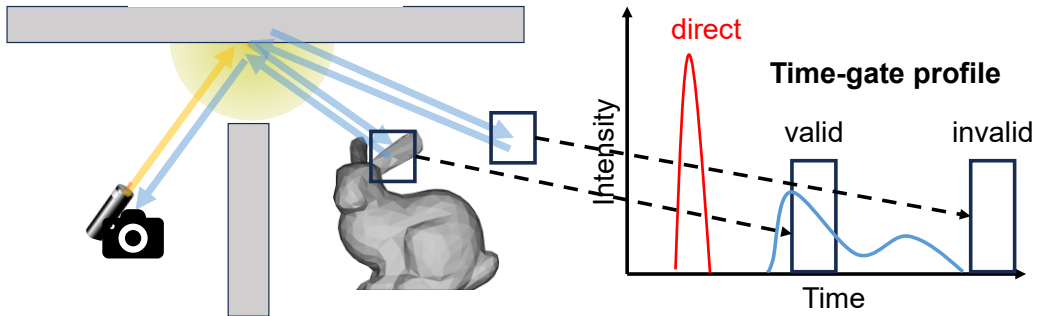
Non-line-of-Sight (NLOS) : *See around the Corner*



36

Some of the light undergoes additional bounces, forming the indirect portion of the transient profile, like this. By analyzing this transient profile directly,

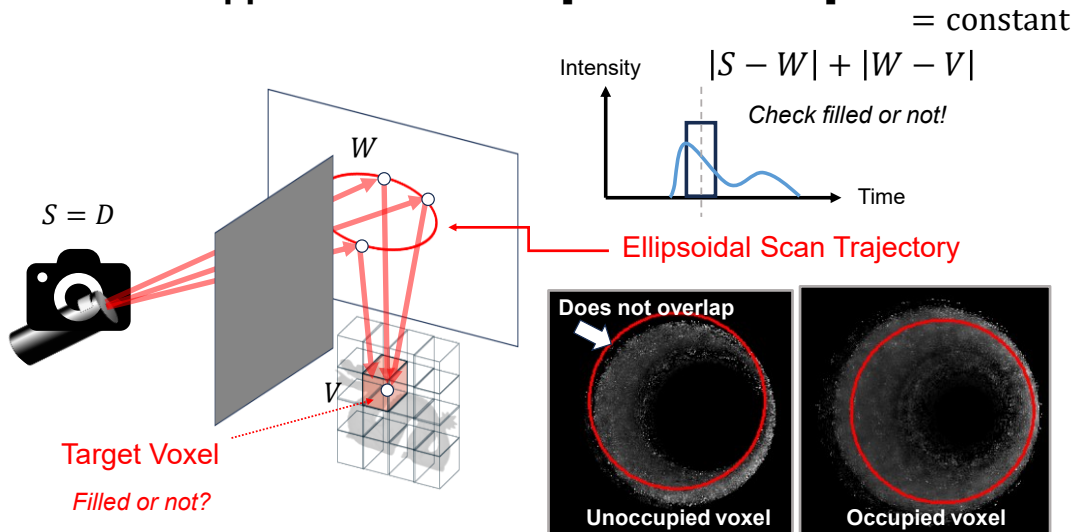
Non-line-of-Sight (NLOS) : *See around the Corner*



37

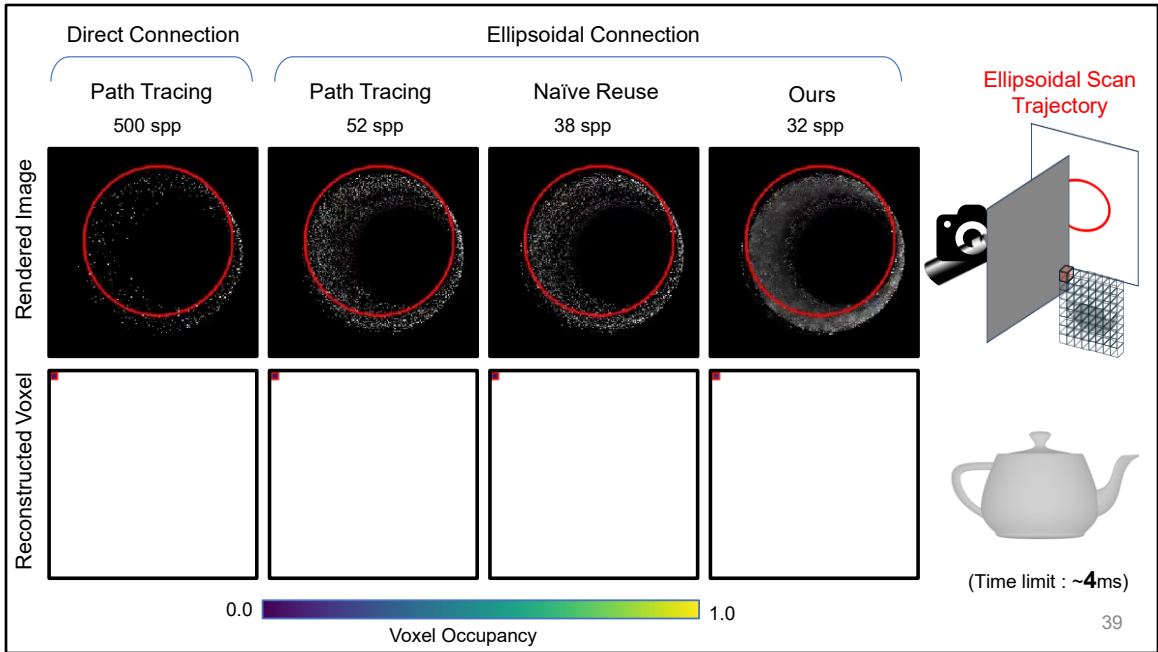
or checking whether a particular time gate contains a valid signal or not, we can infer information about the hidden object.

Downstream Application : SNLOS [Pediredla 2019]

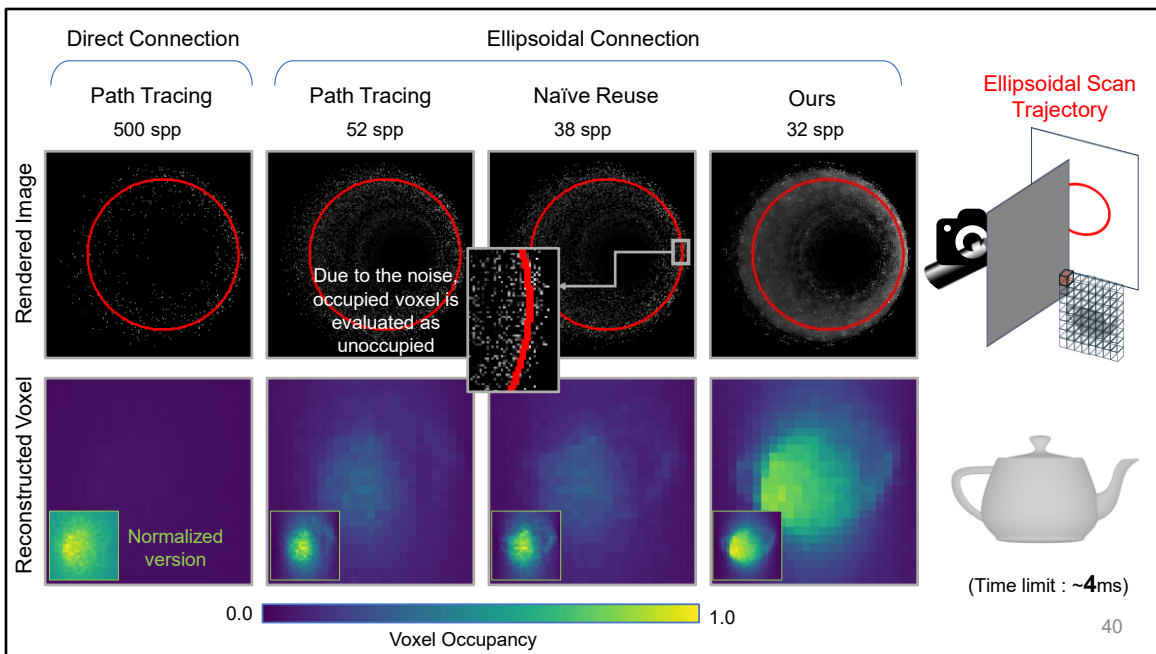


38

Here, we simulate one of the previous NLOS reconstruction works. The goal is to check whether a particular hidden voxel V is occupied. As I explained earlier, this can be done by checking the corresponding time gate contains a valid signal or not. To improve confidence, we can repeat this using different positions on the wall. For a fixed time gate, these positions correspond to an ellipsoid. Therefore, by checking if the measured image aligns with this ellipsoidal trajectory, we can infer that the voxel is occupied or not.

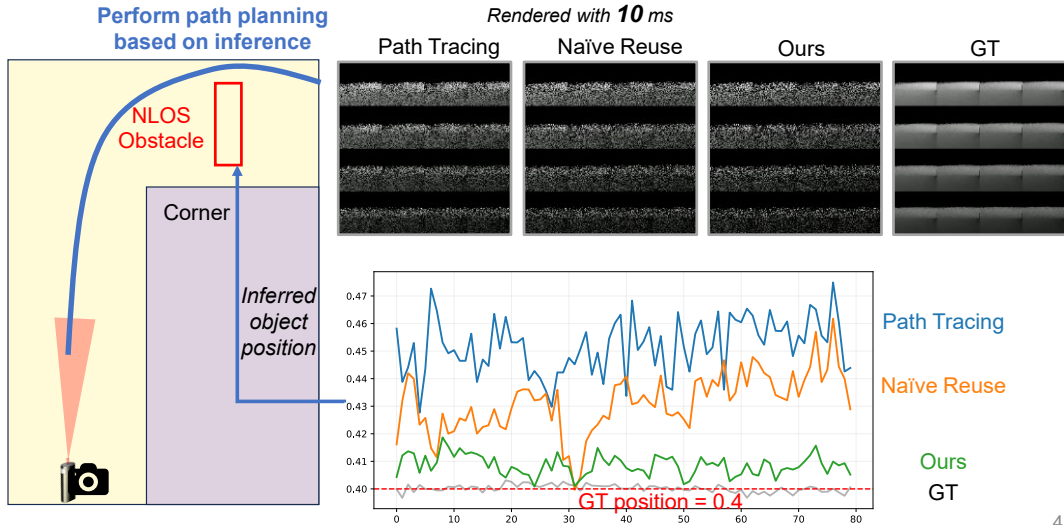


We repeat this process to reconstruct the full 3D object.



As you can see, compared to path tracing or naïve reuse, our method produces more accurate reconstructions thanks to its higher rendering quality. In the baseline methods, Monte Carlo noise often causes truly occupied voxels to be incorrectly classified as empty.

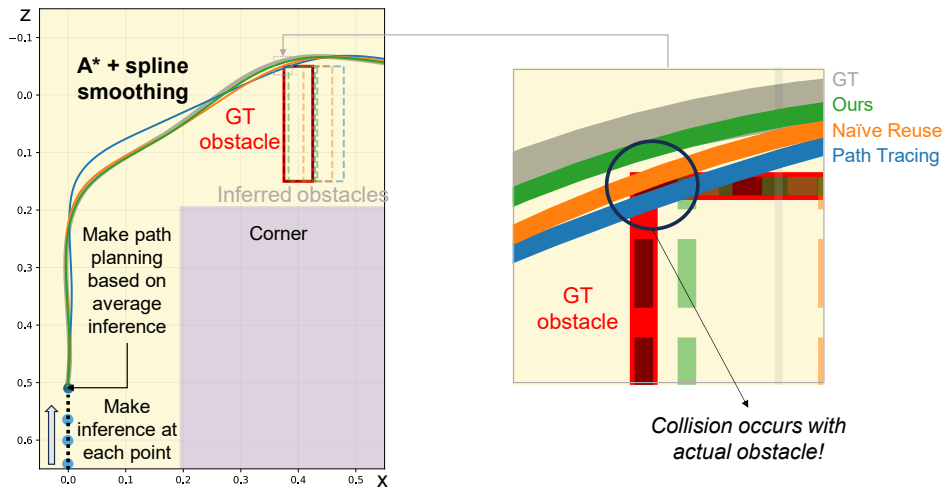
Downstream Application : Navigation [Young 2025]



41

We also simulate an navigation scenario, but I'll leave the details for the poster session.

Downstream Application : Navigation [Young 2025]

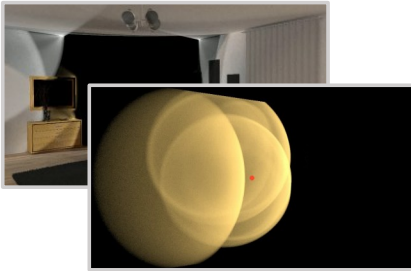


42

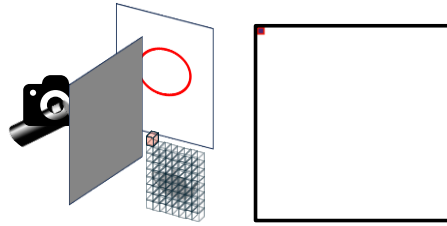
We also simulate an navigation scenario, but I'll leave the details for the poster session.

Conclusion

We propose **path-length-aware shift mapping** that enables aggressive path reuse in ReSTIR even for **ToF rendering**. This allows **fast and robust interactive ToF simulation**.



Interactive-rate ToF Rendering

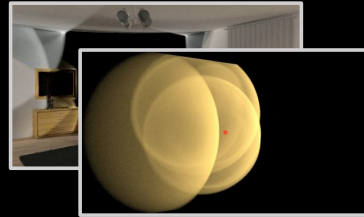


Downstream interactive ToF application

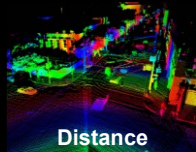
43

In conclusion, we propose a ReSTIR framework with path-length-aware shift mapping that enables interactive time-of-flight simulation, which was not possible with previous methods.

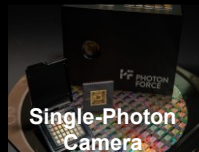
ToF Rendering



Cont. Wave Time-of-Flight Camera



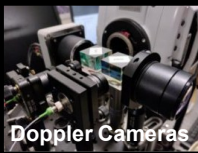
Distance



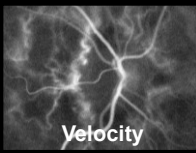
Single-Photon Camera



Transient Profile



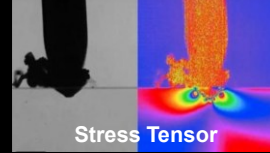
Doppler Cameras



Velocity

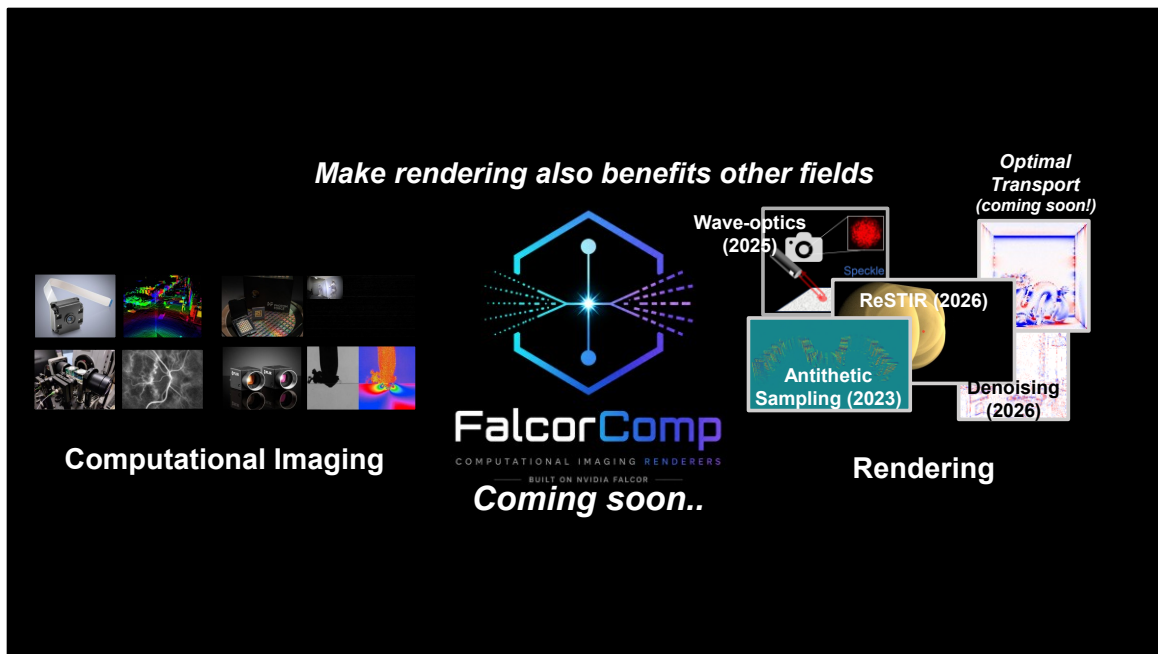


Polarized Camera



Stress Tensor

I'd like to end with a short advertisement. Today, I showed one example how real-time rendering can benefit Time-of-Flight simulation, but the same ideas can impact computational imaging or scientific instrumentation, which requires accurate yet efficient optical simulation algorithm.



That's the motivation behind my research and a comprehensive library that I am working on, FalcorComp, an Falcor-based platform that brings various rendering algorithms to benefit computational imaging and other optical applications. I'm graduating soon and am looking for new opportunities. If this direction seems interesting for please contact. Thank you.